System Architecture Document: KitchenFlow Restaurant Order & Kitchen Display System

Document ID: KF-SAD-001 · Version 0.1 (draft) · 29 October 2026 · Author: Rathi · Related: KF-PDD-001 (Product Development), KF-SRS-001 (Requirements)

1. Introduction

1.1 Purpose

This document describes the architecture that satisfies KitchenFlow's requirements (KF-SRS-001): its structure, technology, runtime behaviour and data, and the cross-cutting concerns that keep it correct and fast under load: request pipeline, caching, rate limiting, concurrency control, security, reliability and observability. It is document 3 of 7. It is written at high-level-design depth, with low-level detail where the design is risky or subtle (sections 5, 6, 9 and 16). Documents 4 (database), 5 (API), 6 (low-level design) and 7 (tests) refine it.

1.2 Scope

In scope: one restaurant outlet, one application server, one PostgreSQL database, browser-based clients and a reverse proxy. Out of scope: multi-outlet or multi-region operation, external integrations and hardware. The scale-out path is described in sections 5.6 and 11 but is not built in version 1.

1.3 Decisions closed by this document

Open issue (SRS section 14)	Decision	Section
OI-02 Client UI technology	React single-page app built with Vite and served by Nginx. Plain HTML and JavaScript against the same API is the fallback.	3.4
OI-03 Database	PostgreSQL	3.4

All other decisions are recorded as ADR-01 to ADR-14 in section 14. Changes this design proposes to the SRS are listed in section 15.2 and applied through the SRS change control.

1.4 Conventions

Figures are referred to as Figure n. Library versions are deliberately not fixed here: pin the current stable releases when the project starts and record them in the README. Limits and timeouts quoted in this document are defaults held in configuration.

2. Architecture drivers

2.1 Quality attributes in priority order

Driver	Source in the SRS	Architectural response
1. Correctness under concurrency: no duplicate, lost or double-updated orders	NFR-CONC-01 to 04, FR-PAY-03, FR-KDS-06, FR-KDS-07	PostgreSQL is the single source of truth; compare-and-set updates, unique constraints, row locks, optimistic versions and an idempotency store (section 6)
2. Real-time visibility within 1 second	NFR-PERF-02, FR-KDS-05	After-commit domain events feed an in-memory SSE hub; clients merge by version and resynchronise on reconnect (section 9)
3. Auditability and access control	FR-AUD-01 to 04, FR-AUTH-01 to 04	Audit written in the same transaction; layered authorization (section 10)
4. Protection against abuse and overload	FR-AUTH-06, RATE_LIMITED, NFR-PERF-01	Two-layer rate limiting, connection caps, explicit timeouts, bulkheads and load shedding (section 5)
5. Fast reads	FR-MENU-06, NFR-PERF-03 and 04	In-process cache and ETags for reference data; indexes and SQL aggregation for lists and reports (sections 5 and 8)

Driver	Source in the SRS	Architectural response
6. Simple to run and explain	NFR-POR-01, PDD goal G6	Modular monolith, three containers, every choice justified by an ADR

2.2 Constraints

Java, Spring Boot and SQL are fixed by the project's purpose. One developer, so every added component must earn its place. No external APIs. Synthetic data only, so there is no customer personal data to protect, but staff credentials and the audit trail still are.

2.3 Load model

Quantity	Value	Basis
Peak orders	150 per hour, about 0.04 per second	SRS section 2.5
API calls per order lifecycle	About 15	Menu read, create, pay, several ticket actions, board refresh, hand-over
Steady peak API traffic	Under 1 request per second	150 x 15 / 3,600 = 0.63 per second, plus screen refreshes
Long-lived connections	About 7 SSE streams	5 stations, manager board, status board
Load-test target	50 concurrent users	NFR-PERF-01; more than 50 times the real peak

Real traffic is tiny. The load tests exist to expose concurrency bugs and prove headroom, not to size hardware. The architecture therefore spends its effort on correctness and protection (limits, timeouts, caps) rather than on horizontal scale.

3. Architecture overview

3.1 Style

KitchenFlow is a **modular monolith**: one deployable Spring Boot application, organised as feature modules with enforced boundaries, layered inside each module. Modules talk through small interfaces and in-process domain events. PostgreSQL is the only stateful component. The application itself keeps only three in-memory structures (the cache, the rate-limit buckets and the SSE connections), and all three are safe to lose on restart.

3.2 Principles

#	Principle	Consequence
P1	The database decides; the cache only displays.	Order validation and pricing always read the database; caches serve screens.
P2	One business operation is one short transaction.	No remote calls and no SSE sends inside a transaction.
P3	State changes only through the state machine.	Controllers and repositories cannot set a status directly.
P4	Push after commit.	Screens never see uncommitted or rolled-back data.
P5	Fail closed for security, fail open for performance aids.	A broken cache or limiter does not take the system down; a broken auth check does.
P6	Every limit and timeout is explicit and configurable.	No unbounded queues, pools or waits.
P7	Idempotent writes.	Retries are safe by design.
P8	Observable by default.	Correlation ID, metrics and structured logs from the first endpoint.
P9	Simple first, with a documented scale path.	No Redis, broker or microservices until a requirement needs them.
P10	Boundaries are tested, not trusted.	An architecture test enforces module rules (NFR-MNT-01).

3.3 Technology stack

Layer	Choice	Reason
Language and runtime	Java 21 (17 minimum)	LTS release; records and modern syntax
Application framework	Spring Boot 3.x: Web MVC, Validation, Security, Data JPA, Actuator	Standard in enterprise Java
Database	PostgreSQL 16 or newer	Row locks, partial indexes, percentile functions
Migrations	Flyway	Versioned and repeatable scripts
Connection pool	HikariCP	Spring Boot default; fast and observable
In-process cache	Caffeine through Spring Cache	Simple, fast, atomic per-key loading
Rate limiting	Bucket4j (in-memory token buckets) plus Nginx limit_req	Two layers without extra infrastructure
Real-time	Server-sent events with SseEmitter	One-way push over plain HTTP
Authentication	Spring Security with JWT (resource-server support)	Stateless, role claims
API documentation	springdoc-openapi	Generated from the code
Edge	Nginx as reverse proxy, TLS terminator and static host	Edge limits, SSE-safe proxying, serves the UI
Client	React with Vite and a fetch-based SSE client	Component model suits live boards
Testing	JUnit 5, Mockito, Testcontainers, ArchUnit, k6	Real database, architecture rules, load
Observability	Logback JSON logs, Micrometer with Prometheus format, optional Grafana	Standard Spring tooling

Layer	Choice	Reason
Packaging and CI	Docker Compose, GitHub Actions	One-command run, automatic checks

3.4 Closed decisions

UI (OI-02): React single-page app served by Nginx.

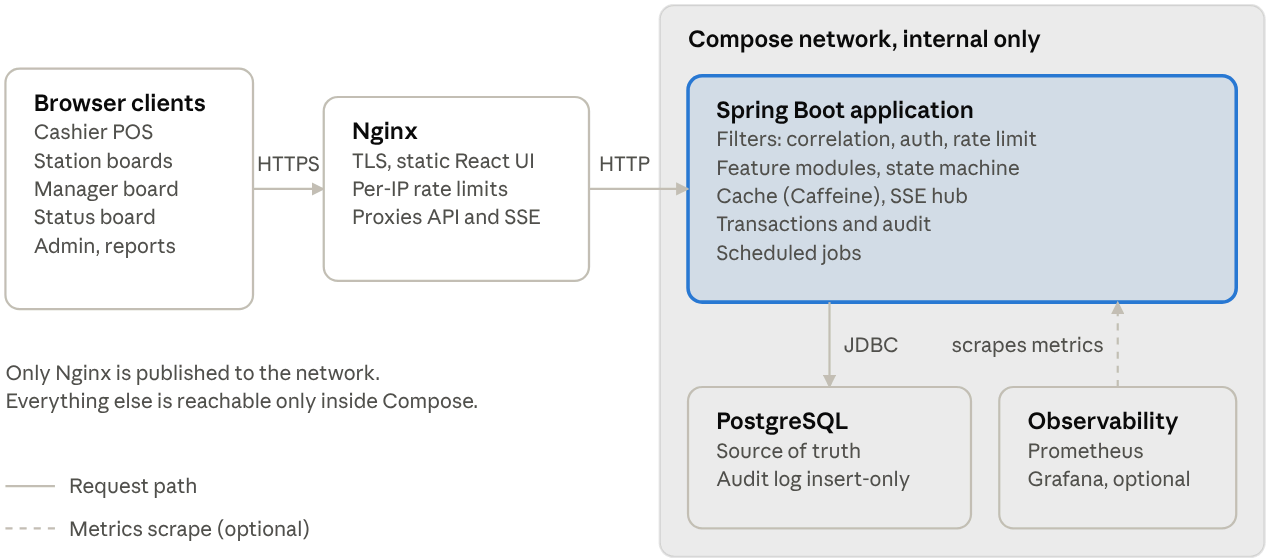
- The SRS already specifies a token-based REST API, CORS and generated OpenAPI. One API can then serve the web screens now and an Android kitchen display later.
- The live boards need client-side state and event handling whatever framework is used; server-rendered pages would duplicate that logic and mix session authentication with the JWT API.
- Fallback: if time is short, build the same screens in plain HTML and JavaScript (fetch plus a streaming reader). The API, the stream contract and the rest of this architecture do not change.

Database (OI-03): PostgreSQL.

- Row-level locking (SELECT FOR UPDATE, SKIP LOCKED), partial indexes, RETURNING and percentile_cont map directly onto FR-KDS-06, FR-ORD-12 and FR-RPT-03.
- Strict constraints and transactional DDL support the integrity rules in SRS section 9.
- MySQL 8 would work for most requirements, but percentile reporting and partial indexes need workarounds.

3.5 Container view

Browser clients (cashier POS, station boards, manager board, status board and admin screens) reach the system through **Nginx**, which terminates TLS, serves the built UI as static files, applies coarse per-client rate limits and proxies /api to the **application** container. The application holds all business logic and talks to **PostgreSQL** over JDBC. An optional **Prometheus** and **Grafana** pair scrapes the application's metrics endpoint. Only Nginx is published to the network; the application and the database are reachable only inside the Compose network. Figure 1 shows the containers and their connections.



container view · 5 containers, 4 connections

Figure 1. Container view. Requests flow from the browsers through Nginx to the application and on to PostgreSQL; the optional observability pair pulls metrics from the application.

4. Logical architecture

4.1 Modules

Module (package)	Responsibility	Owns tables	Events raised
auth	Users, login, JWT, lockout, station binding	app_user, user_station, refresh_token	None
menu	Categories, items, modifiers, combos, availability, menu version	category, menu_item, item_station, modifier_group, modifier, combo	MenuChanged
inventory	Stock levels, reservations, adjustments	item_stock, stock_adjustment	StockDepleted

Module (package)	Responsibility	Owns tables	Events raised
order	Order lifecycle, pricing, tokens, idempotency, cancellation	orders, order_line, order_line_modifier, token_counter, idempotency_record	OrderStatusChanged
payment	Payment and refund records, bill	payment, refund	None
kitchen	Tickets, station queues, start, bump, recall, READY detection	ticket, ticket_item	TicketCreated, TicketChanged, TicketCancelled
board	Public status board snapshot and stream	None (reads only)	None
report	SQL read models for the reports	None (reads only)	None
audit	Append-only event log and its search	audit_event	None
config	Admin-managed settings	app_config	ConfigChanged
platform	Error envelope, correlation ID, rate limiting, cache setup, SSE hub, scheduling, security configuration	None	None

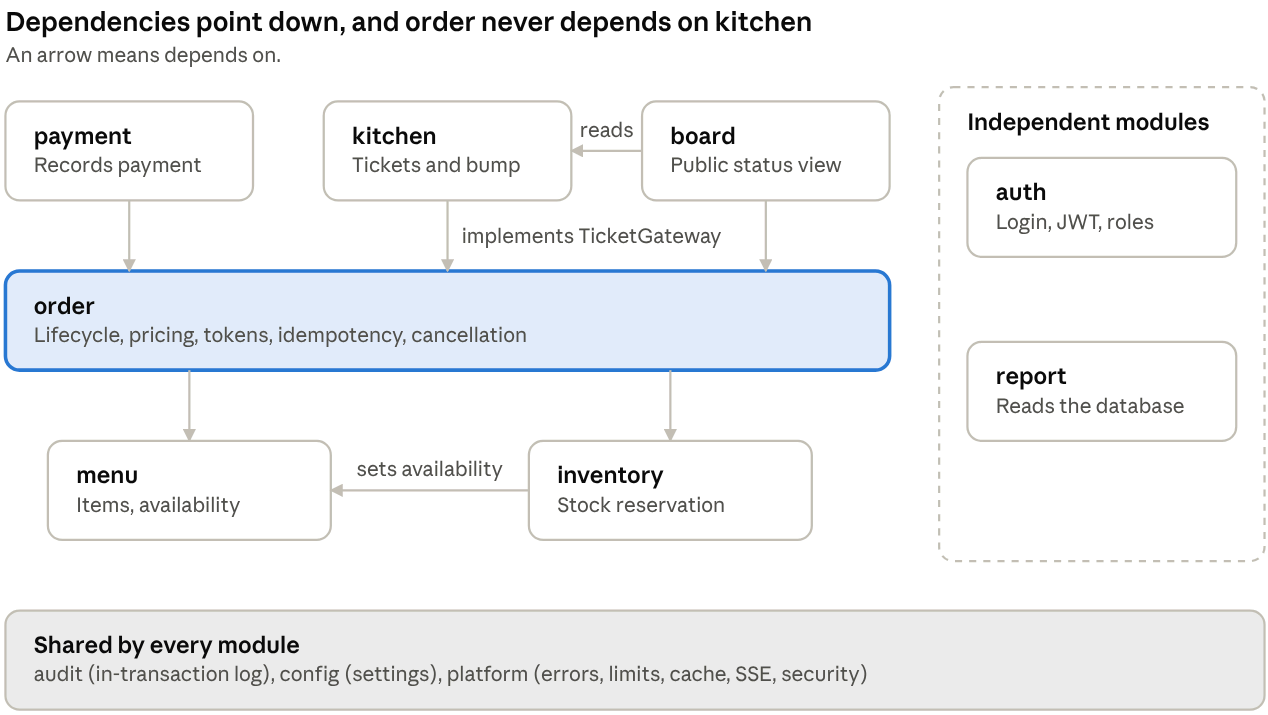
4.2 Dependency rules

A module exposes only its api package. No module touches another module's tables, except report, which reads through SQL. Dependencies point one way (Figure 2):

- payment depends on order.

- kitchen depends on order's public port to read and change order status, and implements order's TicketGateway interface. This inversion lets order create and cancel tickets inside the payment transaction (FR-PAY-02) without order depending on kitchen.
- order depends on menu, inventory and audit.
- inventory depends on menu, to mark an item unavailable at zero stock.
- board depends on order and kitchen for read-only queries.
- Every module may use audit, config and platform; report depends on the database only.

Forbidden: dependency cycles, kitchen depending on payment, menu depending on order, and any controller touching a repository. An ArchUnit test enforces these rules (NFR-MNT-01).



module dependencies · 7 modules, 3 shared

Figure 2. Module dependencies. Order is the hub; kitchen reaches it through a port and implements its TicketGateway interface.

4.3 Layers inside a module

Layer	Contains	Rules
api	Controllers, request and response records, mappers	Validates input shape only; no repositories and no business rules
application	Use-case services, transaction boundary, orchestration, method-level authorization	The only layer that opens transactions
domain	Entities, value objects such as Money, state machines, pricing and rules, domain events	Business decisions live here; no web types
infrastructure	Repositories, native SQL, adapters that implement ports	No business decisions

4.4 Domain events

Event	Raised by	Delivery	Consumers
OrderStatusChanged	order	After commit	Board hub, manager hub
TicketCreated, TicketChanged, TicketCancelled	kitchen	After commit	Station hubs, manager hub
MenuChanged	menu	After commit	Menu cache eviction
StockDepleted	inventory	In the transaction	menu (mark item unavailable)
ConfigChanged	config	After commit	Config cache eviction, station hubs

Rules:

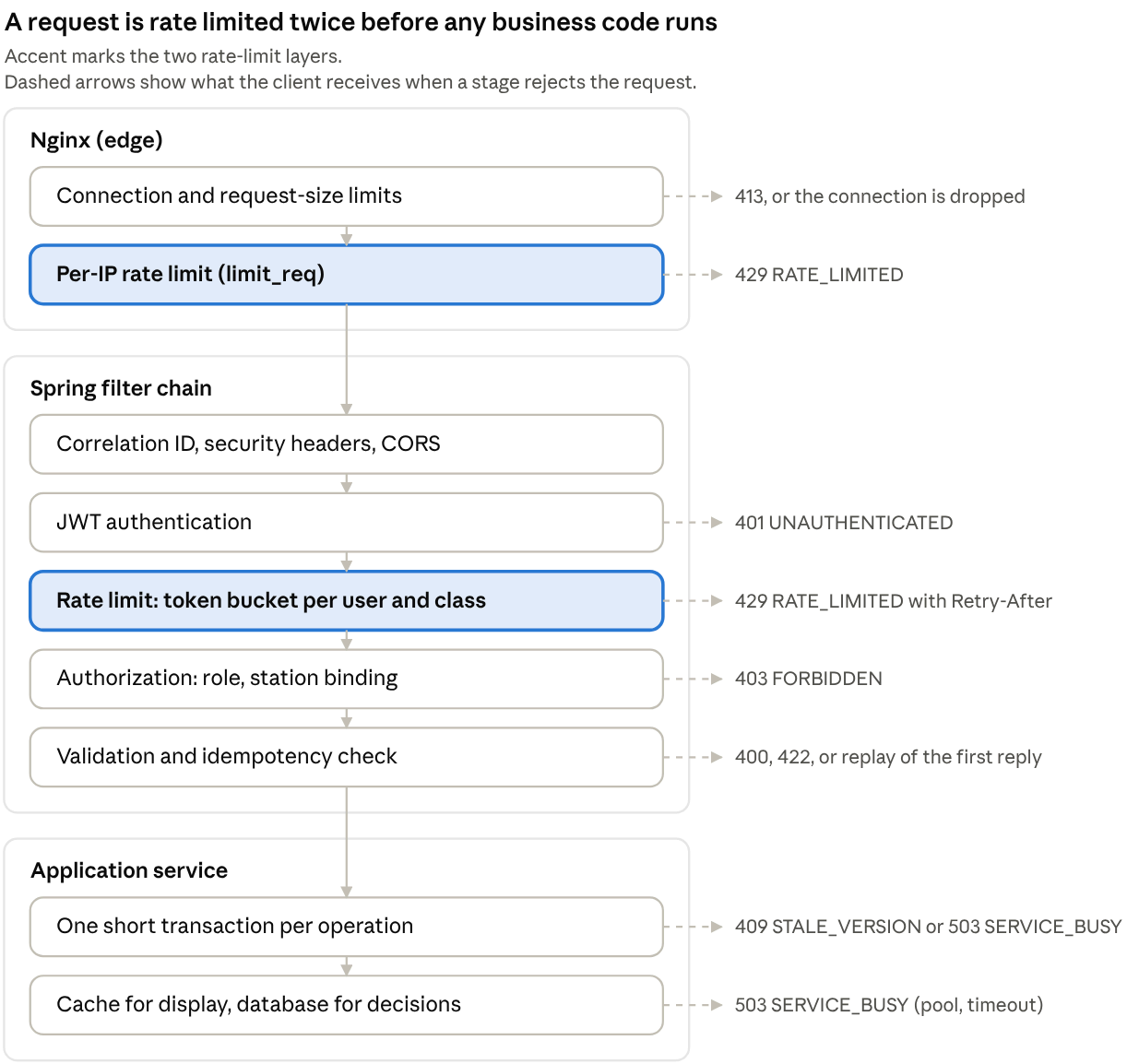
- Audit is not event-driven. AuditService.record() is a direct call inside the business transaction, so a failed audit write rolls the change back (FR-AUD-04).
- After-commit listeners use Spring's transactional event support, run on a bounded executor and never throw into the business flow.

- Delivery to screens is at-most-once. Correctness never depends on it, because clients resynchronise (section 9).
- An outbox table is the scale-out path if events ever need guaranteed delivery (section 5.6).

5. Request pipeline, caching, rate limiting and load handling

5.1 Request pipeline

Every request passes the same ordered stages (Figure 3). Cheap, context-free checks run first, so that hostile or runaway traffic is rejected before it costs a database connection: size and connection limits and the per-IP limit at the edge, then correlation, authentication, per-user rate limiting, authorization and validation in the application. Rate limiting runs after authentication so that its key is the user, not just an address; login, which has no user yet, is limited by address and username. Business code, and the only database work, begins at stage 8.



request pipeline · 9 stages, 2 rate-limit layers

Figure 3. The request pipeline. The accent stages are the two rate-limit layers; dashed arrows show what the client receives when a stage rejects the request.

5.2 Rate limiting

Goals: stop brute-force login, contain retry storms and buggy clients, keep staff screens responsive, bound SSE resources, and never block legitimate peak traffic (under 1 request per second, section 2.3).

Two layers

- **Edge (Nginx limit_req and limit_conn):** coarse, per client address, protects the application process. Behind Docker the client address must come from the trusted proxy header only (set_real_ip_from); otherwise a client could spoof its way around the limit.
- **Application (Bucket4j token buckets):** fine-grained, per user and endpoint class, with burst capacity and steady refill. Buckets live in a Caffeine cache that expires idle entries after 10 minutes and caps the entry count at 10,000, so a flood of distinct keys cannot exhaust memory.

Class	Key	Burst	Refill	Applies to
LOGIN	Address and username	5	5 per 10 minutes	POST /auth/login
REFRESH	User	10	10 per 10 minutes	POST /auth/refresh
ORDER_WRITE	User	30	30 per minute	Create, edit, cancel, pay, complete
TICKET_ACTION	User	120	120 per minute	Start, bump, recall
READ	User	300	300 per minute	Menu, orders, tickets
REPORT	User	10	10 per minute	Reports and audit search
ADMIN_WRITE	User	60	60 per minute	Admin changes
BOARD_READ	Address	60	60 per minute	Board snapshot
SSE_CONNECT	User or address	10	10 per minute	Opening a stream

Additional limits: at most 3 open streams per user, 20 per address and 50 in total. At the edge: 20 requests per second per address with a burst of 40 for /api, and 10 per minute with a burst of 5 for the login path. All values are defaults held in configuration.

Sanity check against the workload. A cashier creates about 2.5 orders per minute at the 150-per-hour peak, so ORDER_WRITE at 30 per minute is 12 times that. Legitimate traffic never touches a limit; a stuck client in a retry loop hits it within seconds.

Login protection. The LOGIN bucket throttles attempts, and the lockout in FR-AUTH-06 (5 failures in 10 minutes, locked for 15) is stored on the user row so it survives restarts. Both return the same generic message as a wrong password where applicable.

Response contract. HTTP 429 with error code RATE_LIMITED, a Retry-After header in seconds and X-RateLimit-Remaining. Clients back off with jitter and never retry a write without its Idempotency-Key.

Exemptions and safety. The health endpoint, static files and SSE heartbeats are not limited. A loadtest profile raises the limits for k6 runs; the application refuses to start with that profile when the production profile is active.

Failure behaviour. The limiter has no external dependency. If it throws an unexpected error, requests fail open with an error log and a metric, except LOGIN, which fails closed.

Observability and tests. A counter kf_rate_limit_rejected_total by class, and an alert when more than 5 percent of requests are rejected. Tests: the sixth login attempt returns 429 even with the right password; buckets refill; one user′s limit does not affect another′s; the load test at design workload shows zero 429 responses.

5.3 Caching

Rule: cache for display and reference data; the database decides. Never cached: orders, tickets, payments, the stock figures used for decisions, and the audit log. Order validation and pricing always read current prices and availability from the database inside the transaction (BR-02, BR-04), so a stale cache can make a screen briefly out of date but can never sell an unavailable item or a wrong price. Configuration is the one cached decision input: it is evicted on change and expires after 60 seconds as a backstop.

Cache	Content	Expiry	Size	Invalidation
menuView	POS menu: categories, items, modifier groups, availability	5 minutes after write	1 entry	Evicted after commit on MenuChanged
stationList	Stations for screens and filters	10 minutes after write	Small	Evicted on ConfigChanged
appConfig	Thresholds, recall window, business-day cutoff, timeouts	60 seconds after write	Small	Evicted after commit on ConfigChanged

HTTP caching. GET /menu returns an ETag derived from a menu version counter that increments on every MenuChanged, with Cache-Control: no-cache, so screens revalidate cheaply and receive 304 when nothing changed. Built UI files have content-hashed names and Cache-Control: public, max-age=31536000, immutable; index.html is no-cache.

Stampede and race handling. Caffeine's loading cache loads a key once even when many requests miss together. An eviction issued after commit runs after any in-flight load of that key finishes, because Caffeine computes each key atomically; a loader that read data from before the commit therefore cannot leave a stale entry behind. A race test in document 7 proves this, and the 5-minute expiry is the backstop.

Staleness budget. The server reflects an availability change in the menu response as soon as the transaction commits (FR-INV-03). POS screens revalidate with the ETag whenever a new order is started and every 15 seconds.

Measurement and sizing. Hit ratio, load time and size are exported through Micrometer; the target hit ratio for menuView is above 95 percent. A menu of 100 items is well under 1 MB, so the three caches together use a few megabytes.

5.4 Load handling and backpressure

Principle: bound everything, reject early and cheaply, and protect the order and ticket path first.

Control	Default	Protects against
Tomcat max threads and accept count	100 and 100	Thread explosion and an unbounded connection queue
Nginx request timeout	15 s for /api; SSE exempt	Hanging requests
Request body size	1 MB	Oversized payloads
Page size	Maximum 100	Huge result sets
HikariCP pool	Maximum 20, minimum idle 5, connection timeout 2 s, maximum lifetime 30 min	Database overload; waits beyond 2 s return 503 SERVICE_BUSY

Control	Default	Protects against
PostgreSQL statement_timeout	5 s for the application role; 10 s for report queries	Runaway queries
PostgreSQL lock_timeout	2 s	Waiting forever on a row lock
idle_in_transaction_session_timeout	30 s	Leaked transactions that hold locks
Report bulkhead	2 concurrent report queries; others wait up to 100 ms, then get 429	Reports starving the order path
SSE sending	Bounded executor; per-subscriber queue of 100; overflow closes the stream	A slow client holding memory

Load-shedding order. Reports are shed first, then reads. Order creation, payment and ticket actions have their own limits and are never queued behind reports.

Capacity reasoning. An order-creation transaction should use about 60 ms of database time, so a 20-connection pool supports roughly 330 of them per second. Token allocation serialises on one counter row held for about 10 ms, so creation tops out near 100 orders per second. The design peak is 0.04 per second, more than 2,000 times below that ceiling. The load test's 50 concurrent users run at the ceiling, over 1,000 times the real peak, which is what exposes locking mistakes.

Latency budget for order creation (NFR-PERF-01: 300 ms at p95).

Step	Budget
Edge, filters, authentication, rate limit	15 ms
Validation and one batched load of items and modifiers	20 ms
Pricing and GST	5 ms
Idempotency insert and stock reservation	30 ms

Step	Budget
Token allocation, order and line inserts (batched)	40 ms
Audit insert	10 ms
Commit	20 ms
Response mapping and headers	10 ms
Total	**150 ms, half of the 300 ms target**

5.5 Query efficiency

- No N+1 queries: fetch joins for an order with its lines, DTO projections for lists, and a test that counts statements per endpoint.
- Hibernate batching (batch size 50, ordered inserts) for order lines and ticket items.
- Offset pagination up to 100 rows per page; keyset pagination for deep order history.
- Reports aggregate in SQL over indexed ranges (section 8.5), inside read-only transactions with the report statement timeout.

5.6 Scale-out path (not built in version 1)

Need	Change
More than one application instance	Shared cache (Redis) or short expiries; Bucket4j on a shared store; SSE fan-out through PostgreSQL LISTEN and NOTIFY or Redis pub/sub
Heavy reporting	Read replica and a materialised daily summary
Guaranteed event delivery	Transactional outbox table with a relay
Several outlets	An outlet id on every table and stream; per-outlet token counters
Higher write rates	Connection pooler (PgBouncer); shard the token counter

None of these is needed for the design workload; the list shows where the architecture would bend, which is the useful answer to a scaling question in an interview.

6. Concurrency and transaction design

6.1 Principles

- PostgreSQL's default READ COMMITTED isolation, plus explicit locks where an invariant spans rows. SERIALIZABLE is rejected: it needs retry handling on every write path, costs throughput and is harder to explain.
- Optimistic locking (a version column) for a single row edited by people, such as a ticket; compare-and-set updates for state and counters; pessimistic row locks (SELECT FOR UPDATE) for invariants that span rows, such as an order and its tickets; unique constraints as the last line of defence.
- Short transactions: no network calls and no SSE sends inside one; events leave only after commit.
- One global lock order: the order row, then its tickets by id, then stock rows by item id. A fixed order prevents deadlocks.

6.2 Invariants and their mechanisms

Invariant	Mechanism	On violation	Verified by
One order per idempotency key	The idempotency row is inserted first, inside the order's own transaction, with a unique key per user. A concurrent duplicate blocks on the unique index until the first transaction commits, then reads the stored response and replays it. If the first rolls back, the duplicate proceeds. A payload hash mismatch is rejected.	422 IDEMPOTENCY_KEY_REUSED	NFR-CONC-01, AT-03

Invariant	Mechanism	On violation	Verified by
Unique token per business day	A counter row per business day is incremented with UPDATE … RETURNING, allocated as late as possible to shorten the lock; unique (business_day, token) as backstop	None expected	FR-ORD-04
No overselling	Conditional UPDATE: stock = stock - n where stock >= n, in ascending item order; zero rows updated rolls back the whole transaction, so there is no partial reservation	409 OUT_OF_STOCK	NFR-CONC-02, AT-04
Exactly one payment	Compare-and-set UPDATE orders SET status = PAID WHERE id = :id AND status = CREATED, which must update one row; unique (order_id) on payment	409 PAYMENT_ALREADY_RECORDED	FR-PAY-03
One writer per ticket	JPA @Version on ticket; the client sends the version it saw	409 STALE_VERSION	NFR-CONC-03, AT-05

Invariant	Mechanism	On violation	Verified by
READY exactly once	Every bump, recall and cancel locks the order row first (section 6.3)	None expected	NFR-CONC-04, AT-06
Cancel versus progress	The same order lock; the state guard is checked after the lock, so the loser sees the new state	409 INVALID_TRANSITION	AT-07
Audit with the change	Same transaction; a failed audit write rolls back the change	500 and rollback	AT-11, FR-AUD-04
Unpaid timeout versus payment	The job selects candidates with FOR UPDATE SKIP LOCKED, then applies the same compare-and-set	Job skips the order	FR-ORD-12

6.3 Why READY needs an order-level lock

Two cooks bump the last two tickets of an order at the same moment. Without a lock, each transaction updates its own ticket row and then checks whether any ticket is still open. Under READ COMMITTED each sees the other's ticket as still in progress, because the other transaction has not committed. Neither sets the order READY, and it sits in IN_KITCHEN forever.

The fix is to lock the order row (SELECT … FOR UPDATE) at the start of every bump, recall and cancel. The second transaction waits until the first commits, then sees the first ticket DONE, counts zero open tickets and sets READY exactly once. Locking the order before the ticket gives a single lock order and so no deadlock. Describing the bug first and the fix second is the strongest way to tell this in an interview.

6.4 Transaction boundaries

Operation	Transaction scope	Locks, in order	Retry
Create order	Idempotency row, validation, pricing, stock reservation, token, order and lines, audit	Idempotency row, stock rows by item id, token counter row	Deadlock retry, at most 2
Edit order (CREATED)	Recalculate, adjust stock, audit	Order row, stock rows by item id	Deadlock retry, at most 2
Record payment	Order compare-and-set, payment, tickets, order to IN_KITCHEN, audit	Order row	None
Start ticket	Version check, state change, audit	Ticket row (version)	None
Bump or recall	Order lock, ticket change, READY evaluation, audit	Order row, then ticket row	None
Cancel order	Order lock, tickets, stock restore (CREATED only), refund entry, audit	Order row, ticket rows by id, stock rows by id	Deadlock retry, at most 2
Report	Read-only transaction with statement timeout	None	None

6.5 Deadlocks, timeouts and retries

- lock_timeout of 2 seconds: a statement that waits longer fails with SQLSTATE 55P03 and returns 503 SERVICE_BUSY with Retry-After (a proposed SRS error code, section 15.2).
- Deadlock (40P01): the whole transaction is retried at most twice with a 20 to 60 ms jittered delay, only for operations that are safe to repeat.
- Optimistic conflicts are never retried automatically: the user decides, with the current state in the 409 response.
- Retries are counted in kf_db_lock_retry_total.

6.6 Proving it

Concurrency tests (NFR-CONC-01 to 04, AT-03 to AT-06) run against real PostgreSQL through Testcontainers. N threads are released together by a CountDownLatch and the assertions read the database: one order, stock exactly zero, one READY audit record. Each test repeats 50 times to expose flakiness. To prove the tests can fail, a mutation check removes each lock or constraint in turn and confirms that the matching test goes red.

7. Key runtime flows

7.1 Create an order

Step	Component	Action
1	Browser	POST /orders with an Idempotency-Key
2	Nginx and filters	Edge limits, correlation ID, JWT, ORDER_WRITE bucket, role check
3	Controller	Bean Validation of the request shape
4	OrderService	Begin the transaction
5	IdempotencyService	Insert the key row; a concurrent duplicate blocks here and later replays
6	MenuQuery	Load items, modifiers and availability from the database in one batched query, never from the cache
7	OrderService	Validate availability, quantity, modifier minimum and maximum; collect all errors
8	PricingService	Price lines and GST with BigDecimal
9	StockService	Reserve stock with conditional updates in item-id order
10	TokenService	Allocate the token just before the insert
11	OrderRepository	Insert order, lines and modifiers as a batch
12	AuditService	Record ORDER_CREATED

Step	Component	Action
13	IdempotencyService	Store the response with the key, then commit
14	Controller	201 with totals; metrics recorded

7.2 Record payment and send to the kitchen

Step	Component	Action
1	Filters	As above, with the ORDER_WRITE bucket
2	PaymentService	Begin the transaction
3	OrderWorkflow	Compare-and-set CREATED to PAID; zero rows means 409
4	PaymentRepository	Insert the payment; the amount must equal the order total
5	TicketGateway (kitchen)	Group lines by station using the item-to-station mapping from the database; insert tickets and ticket items
6	OrderWorkflow	Compare-and-set PAID to IN_KITCHEN
7	AuditService	Record payment, tickets and status change
8	Transaction	Commit
9	Event listeners (after commit)	TicketCreated to each station hub; OrderStatusChanged to the board and manager hubs
10	Controller	201 with the bill

7.3 Bump a ticket to READY

Step	Component	Action
1	Filters	TICKET_ACTION bucket; station binding check
2	TicketService	Begin the transaction; read the ticket's order id

Step	Component	Action
3	Order lock	SELECT … FOR UPDATE on the order row
4	TicketService	Re-read the ticket; compare its version with the client's; mismatch returns 409 STALE_VERSION
5	State machine	IN_PROGRESS to DONE
6	TicketService	Count open tickets; if zero and the order is IN_KITCHEN, compare-and-set to READY
7	AuditService	Record the changes, then commit
8	Event listeners (after commit)	TicketChanged to station and manager hubs; OrderStatusChanged to the board when READY

7.4 A station screen connects and reconnects

1. The client opens the authenticated stream and starts buffering events.
2. The server checks role and station binding, registers a subscriber and sends a hello event carrying server time.
3. The client fetches the snapshot of the station's open tickets.
4. The client merges the snapshot, then buffered and live events, keeping the highest version of each ticket.
5. The server sends a heartbeat every 15 seconds.
6. On disconnect the client reconnects with exponential backoff (1, 2, 4, 8 seconds, then 15 at most, with jitter) and repeats steps 1 to 4.
7. Safety nets: a full snapshot refresh every 60 seconds, and a 2-second snapshot poll while the stream is down.

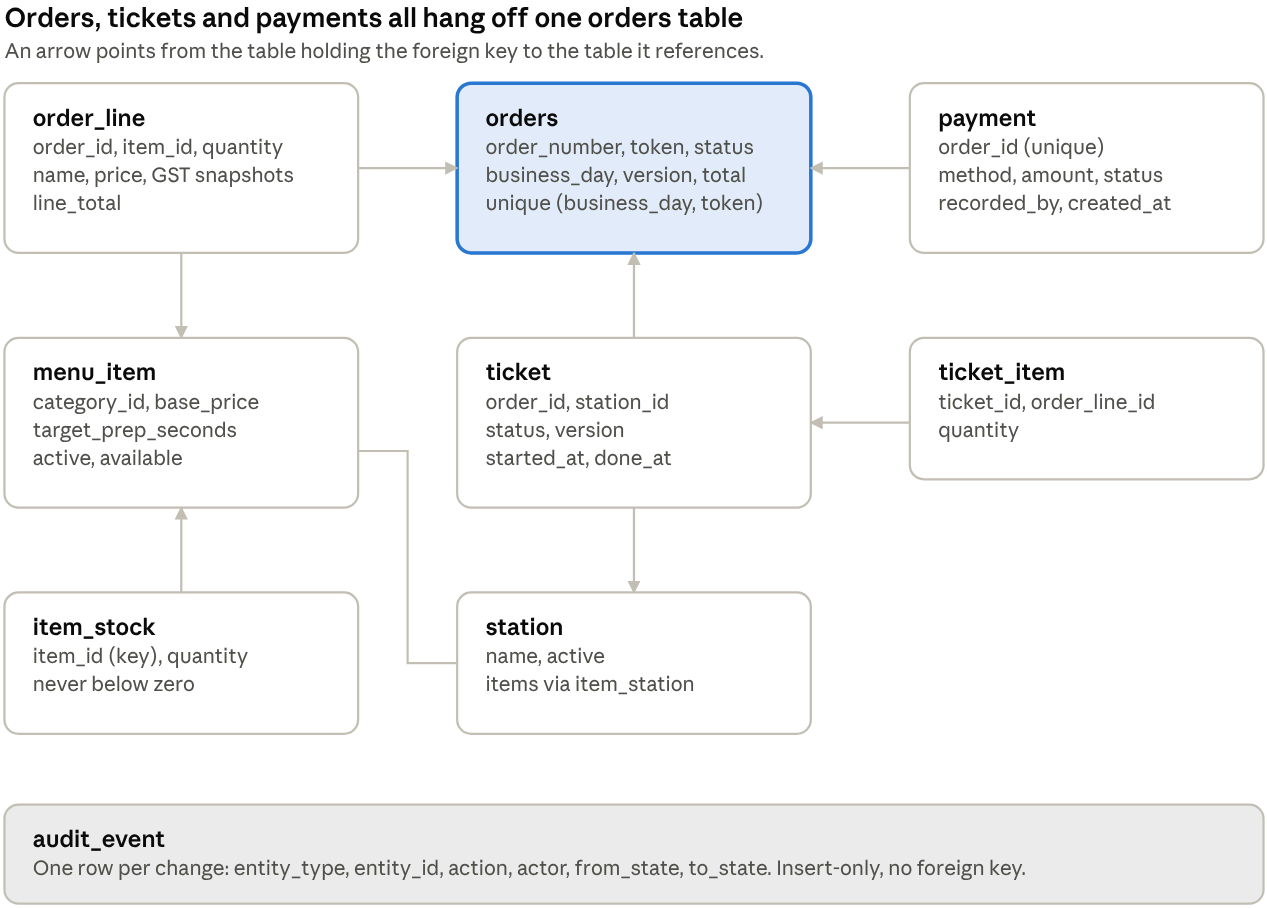
8. Data architecture

8.1 Overview

One PostgreSQL database with three roles: kf_migrator (schema changes, used only by Flyway at start-up), kf_app (data changes only; the application's runtime role) and an optional read-only role for report experiments. On audit_event, kf_app has INSERT and SELECT only, and a trigger rejects UPDATE and DELETE as defence in depth (FR-AUD-02).

8.2 Logical model

Figure 4 shows the main tables. Money is NUMERIC(12,2) and timestamps are TIMESTAMPTZ stored in UTC. Order lines carry snapshots of name, price and GST rate. Each ticket belongs to one order and one station. Categories, modifiers, combos, users, the token counter and the idempotency record follow the SRS section 9 entities and are specified in document 4.



logical data model · 8 tables and the audit log

Figure 4. Logical data model. Orders are the hub; the audit log references any entity by type and id.

8.3 Constraints and indexes (confirmed with EXPLAIN in document 4)

Structure	Definition	Serves
Unique constraints	order_number; (business_day, token); payment (order_id); idempotency (user, key)	BR-01, FR-PAY-03, BR-15
Check constraints	quantity 1 to 50; price above 0; stock 0 or more; status values	DR-03
Foreign keys	On every relationship	DR-03
Index on orders	(status, created_at DESC)	Open orders and search
Index on orders	(business_day, status)	Reports and tokens
Partial index on ticket	(station_id, created_at) WHERE status is NEW or IN_PROGRESS	Station queue; small and fast
Index on ticket	(order_id)	READY check
Indexes on order_line	(order_id) and (item_id)	Order loading; top-items report
Indexes on audit_event	(entity_type, entity_id, created_at DESC) and (created_at)	Audit search
Index on idempotency_record	(created_at)	24-hour purge

8.4 Access patterns

Use case	Query shape	Target
Station queue	Open tickets of one station, rush first, then oldest	Milliseconds; fewer than 50 rows
READY check	Count open tickets of one order	A handful of rows
POS menu	Whole menu, once per cache window	Served from cache
Order search	Filter by status, date, token; page of 20 to 100	p95 under 300 ms at 100,000 orders

Use case	Query shape	Target
Sales report	Aggregate paid orders by business day, payment method	p95 under 2 s
Preparation times	percentile_cont of done minus started, by station and item	p95 under 2 s
Audit search	Filter by entity, actor or date	p95 under 300 ms at 500,000 rows

8.5 Reporting

Reports aggregate in SQL over indexed ranges of at most 92 days. The business_day column is stored on orders so that date predicates stay indexable, and preparation-time statistics use percentile_cont. Reports run in read-only transactions with the report statement timeout and the report bulkhead (section 5.4). If reports ever slow down, a daily summary table refreshed by a scheduled job is the next step; it is not part of version 1.

8.6 Migrations and seed data

Flyway applies versioned scripts. While the schema is still changing, keep a single baseline migration (V1) and replace it wholesale; after the first demo build, switch to forward-only incremental migrations (V2, V3 and so on). Seed data and the 100,000-order generator (SQL using generate_series) are separate profile-gated commands and never part of migrations.

8.7 Backup and retention

A nightly pg_dump in a Compose profile keeps 7 daily copies, and a restore is tested once before the demo. Targets for the demo: recovery point 24 hours, recovery time 15 minutes. Orders and audit records are kept for at least 12 months (DR-09). The only deletions in version 1 are idempotency records after 24 hours and refresh tokens after expiry.

9. Real-time architecture (SSE)

9.1 Why server-sent events

Screens need one-way push from server to browser; their actions go back as ordinary REST calls. SSE runs over plain HTTP, passes proxies, reconnects automatically and needs no extra protocol. WebSocket would add a second protocol, a different authentication story and manual reconnection for no benefit here (ADR-03).

9.2 Streams

Stream	Audience	Authentication	Events	Snapshot endpoint
/api/v1/stream/stations/{id}	Kitchen (bound to the station), Manager	JWT	TICKET_CREATED, TICKET_CHANGED, TICKET_CANCELLED, CONFIG_CHANGED	GET /api/v1/stations/{id}/tickets
/api/v1/stream/manager	Manager	JWT	ORDER_CHANGED, TICKET_CHANGED, LATE_TICKET	GET /api/v1/orders with the open filter
/api/v1/stream/board	Public	None	BOARD_CHANGED	GET /api/v1/board

9.3 Event design

Each event carries the full current state of the object, not a difference, together with its version: for example a ticket with its id, status and version. The client keeps the highest version it has seen for each id, so processing is idempotent, independent of order and tolerant of gaps and duplicates. Events also carry a name and a per-process sequence id for debugging, but correctness never depends on replay.

9.3a Hub design

- The SseHub keeps a concurrent map from stream key to subscribers. Each subscriber wraps an SseEmitter and a bounded queue of 100 events.
- After-commit listeners call publish(); a small bounded executor drains each queue and sends.
- A heartbeat comment every 15 seconds stops proxies closing idle streams and lets the server detect dead clients.

- A subscriber is removed on completion, timeout or error. A subscriber whose queue overflows or whose send fails is closed, never skipped: the client reconnects and resynchronises, so an event is never silently lost.
- Connection caps and the SSE_CONNECT bucket apply (section 5.2). A gauge kf_sse_connections reports open streams by stream type.

9.4 Consistency model

The database is the truth and the push is a hint. Delivery happens after commit and is at-most-once. A screen stays correct because it resynchronises on connect, on reconnect, when told SNAPSHOT_REQUIRED (sent on graceful shutdown) and on a 60-second safety refresh. With a healthy stream, staleness equals the push latency (target under 1 second); with a failed stream the client polls the snapshot every 2 seconds.

9.5 Authenticated streams in browsers

The browser's EventSource cannot send an Authorization header, and putting a token in the URL would leak it into logs. Authenticated streams therefore use a fetch-based SSE client that sends the header; the public board uses plain EventSource. The server validates the token at connect and closes the stream when that token would expire (15 minutes); the client refreshes the token and reconnects. A deactivated user's streams therefore end within 15 minutes, matching FR-AUTH-05.

9.6 Proxy settings

For /api/v1/stream/ Nginx uses HTTP/1.1, proxy_buffering off, proxy_cache off and proxy_read_timeout 3600s, and the application sends X-Accel-Buffering: no. Without these, events arrive in delayed batches, which is the most common SSE deployment bug.

9.7 Multi-instance path

Fan out through PostgreSQL LISTEN and NOTIFY (no new infrastructure) or Redis pub/sub; each instance feeds its own subscribers.

10. Security architecture

10.1 Trust boundaries

Clients are untrusted. Nginx is the edge. The application trusts only what it has validated. The database accepts connections only from the application, over the Compose network, with least-privilege roles.

10.2 Authentication

- Login issues a 15-minute access token (JWT) holding subject, role, bound stations and a token id. The signing key comes from the environment, is at least 256 bits and is never committed.
- Refresh tokens (Should): random, stored hashed, rotated on every use and delivered in an HttpOnly, Secure, SameSite=Strict cookie limited to the auth path. Reuse of a consumed token revokes the whole chain (FR-AUTH-07).
- Passwords use BCrypt with a cost chosen so hashing takes about 250 ms on the target machine. Wrong-user and wrong-password cases take similar time and return one message.
- Lockout counters live on the user row; the LOGIN bucket adds throttling (section 5.2).

10.3 Authorization

Four layers, deny by default: (1) URL rules by role; (2) method security on application services; (3) station binding in the ticket service; (4) the state machine, which checks the actor's role for each transition. AT-08 tests every role against every endpoint.

10.4 Threats and controls

Threat	Example	Control	Requirement
Credential guessing	Brute-force login	BCrypt, lockout, LOGIN bucket, edge limit	FR-AUTH-06
Privilege escalation	A cook calls a manager endpoint	Layered authorization, deny by default	FR-AUTH-03, AT-08
Price tampering	Client sends a lower price	Server prices from the database; snapshots on lines	BR-02

Threat	Example	Control	Requirement
Replay and duplicates	A retry creates a second order	Idempotency key	FR-ORD-10
Injection	Malicious text in a note	Bean Validation, bound parameters, no SQL built from text	NFR-SEC-02
Audit tampering	Editing history	Insert-only grants and a trigger	FR-AUD-02
Denial of service	Request flood, many streams	Edge and application limits, stream caps, timeouts, report bulkhead	Section 5
Information leakage	Stack trace in an error	Error envelope without internals; the board shows tokens only	FR-SYS-01, FR-BRD-01
Token theft	Script reads the token	Token kept in memory only, strict CSP, short expiry	NFR-SEC-01, NFR-SEC-03
Secrets exposure	Key committed to the repository	Environment variables, only .env.example committed, secret scanning in CI	NFR-SEC-01
Vulnerable dependencies	A known flaw in a library	Pinned versions, automated update pull requests, dependency scan in CI (Should)	NFR-SEC-05

10.5 Browser and transport

CORS is limited to configured origins. Headers: Content-Security-Policy with default-src 'self', X-Content-Type-Options nosniff, Referrer-Policy no-referrer and frame-ancestors none. HTTPS is required outside local development. The database port is never published.

11. Reliability and failure handling

11.1 Failure modes

Failure	Detection	Behaviour	Recovery
Database down	Readiness fails; connection timeout	API returns 503; screens show an offline banner and keep polling	Pool reconnects automatically
Pool exhausted	Pending-connection metric; 2-second timeout	503 SERVICE_BUSY with Retry-After	Load drops; investigate with metrics
Slow query	statement_timeout	Request fails and is logged with its correlation ID	Fix the index or query
Lock wait or deadlock	SQLSTATE 55P03 or 40P01	Bounded retry, then 503	Review contention counter
Slow or dead SSE client	Failed send or full queue	Stream closed	Client reconnects and resynchronises
Application crash	Container health check	Docker restarts it; no data is lost because commits are in PostgreSQL	Screens reconnect
Duplicate request or retry	Idempotency key	Stored response replayed	None needed
Device clocks differ	Timers use server timestamps	Ages stay consistent	None needed
Scheduler overlap	SKIP LOCKED	Each order handled once	None needed

11.2 Graceful shutdown

The server shuts down gracefully with a 20-second grace period. On SIGTERM readiness goes down, new requests stop, in-flight transactions finish, and every stream receives SNAPSHOT_REQUIRED before it is closed.

11.3 Health checks and scheduled jobs

Liveness reports that the process is up; readiness reports that the database is reachable and migrations are applied. Compose health checks gate start-up order. Scheduled jobs: the unpaid-order timeout (every minute), idempotency purge and refresh-token purge (hourly). Each is safe with one or many instances because it claims rows with SKIP LOCKED.

11.4 Degraded modes

If this fails	The system
SSE streams	Screens poll the snapshot every 2 seconds
A cache	Loads from the database
The rate limiter (unexpected error)	Fails open, with an error log and metric; login fails closed
The metrics stack	Keeps taking orders; only dashboards are lost

12. Observability

12.1 Logs

Structured JSON to stdout, one line per event, with timestamp, level, correlationId, userId, orderNumber, stationId and message. The correlation ID is accepted from X-Correlation-Id or generated, returned in a response header and included in every error envelope. Passwords, tokens and Authorization headers are never logged. INFO records business events, WARN records rejected and retried operations, ERROR records unexpected failures.

12.2 Metrics

Metric	Type	Purpose
http_server_requests by route and status	Timer	Latency and error rate
kf_order_create_seconds	Timer	NFR-PERF-01
kf_orders_created_total	Counter	Throughput
kf_ticket_cycle_seconds by station	Timer	Kitchen performance
kf_sse_connections by stream	Gauge	Open streams
kf_sse_stream_closed_total by reason	Counter	Overflow, timeout, error
kf_rate_limit_rejected_total by class	Counter	Limiter behaviour
Cache hits, misses and load time by cache	Standard	Cache effectiveness
hikaricp active and pending connections	Gauge	Pool pressure
kf_db_lock_retry_total	Counter	Contention
kf_idempotent_replay_total	Counter	Retry frequency
kf_report_bulkhead_rejected_total	Counter	Report shedding

12.3 Alerts

Examples, written as thresholds on Grafana panels in version 1: order creation p95 above 300 ms for 5 minutes; pending database connections above 0 for 1 minute; 5xx rate above 1 percent; rate-limit rejections above 5 percent; no open stream for a station during opening hours. Automated alert delivery is optional.

12.4 Logs versus audit

Logs are for operators and may be rotated. The audit log is business data: kept for 12 months, stored in the database and written in the same transaction as the change.

13. Deployment and delivery

13.1 Topology

Service	Image	Exposed	Health check	Notes
nginx	Nginx with the built UI	Ports 80 and 443 to the LAN	HTTP 200 on /	Edge limits, TLS, proxy
app	Application image (Java 21, multi-stage build)	Internal 8080; management port internal	Readiness endpoint	Runs as a non-root user
postgres	PostgreSQL 16	Internal 5432 only	pg_isready	Named volume for data
prometheus, grafana (observability profile)	Official images	Internal; Grafana optional on a local port	HTTP	Scrapes application metrics
backup (backup profile)	PostgreSQL client image	None	Exit code	Nightly pg_dump to a volume

13.2 Configuration and profiles

Profiles: dev (application on the host, database in Compose), compose (everything in containers), test (Testcontainers) and loadtest (raised limits, refused alongside production settings). Configuration comes from environment variables. A committed .env.example lists them (database URL and credentials, JWT key, allowed CORS origins, rate-limit and cache settings); real values are never committed. An excerpt of the application configuration:

```
kitchenflow:
  rate-limit:
    login:         { capacity: 5,   refill: 5,   period: 10m }
    order-write:   { capacity: 30,  refill: 30,  period: 1m }
    ticket-action: { capacity: 120, refill: 120, period: 1m }
```

```
    report:        { capacity: 10,  refill: 10,  period: 1m }
    sse:           { per-user: 3, per-address: 20, total: 50 }
  cache:
    menu-view:     { expire-after-write: 5m }
    app-config:    { expire-after-write: 60s }
spring:
  datasource:
    hikari: { maximum-pool-size: 20, minimum-idle: 5, connection-timeout:
2000 }
  lifecycle:
    timeout-per-shutdown-phase: 20s
server:
  shutdown: graceful
  tomcat:
    threads: { max: 100 }
    accept-count: 100
```

13.3 Continuous integration

Stages, in order: checkout, build with Maven, unit tests, integration and concurrency tests with Testcontainers, architecture tests with ArchUnit, static analysis and formatting, coverage gate (at least 70 percent on services and domain), dependency scan (Should), container build, and a smoke test (Compose up, health check, a short k6 script). A failing stage fails the build.

13.4 Environments

Local Compose is the primary environment and the demo. CI runs ephemeral containers. An optional cloud demo runs the same Compose stack on one small virtual machine; check current pricing and set a billing alarm first.

13.5 Build order

Roadmap phases 1 to 3 run only the application and the database (dev profile). Nginx, observability and backup are added in the polish phase, so infrastructure never delays the Must features. The UI is built against the same API from the first working endpoint.

14. Architecture decision records

ID	Decision	Alternatives considered	Rationale	Trade-off
ADR-01	Modular monolith	Microservices; unstructured monolith	One developer; order, payment and kitchen changes must be atomic (FR-PAY-02); enforced boundaries keep a later split possible	Scales vertically only
ADR-02	PostgreSQL	MySQL 8	Row locks, partial indexes, percentile functions, RETURNING	Less common in some shops; the concepts transfer
ADR-03	SSE for real-time push	WebSocket; polling only	One-way push over plain HTTP with automatic reconnect; polling stays as the fallback	One direction only; needs a fetch-based client for authentication
ADR-04	Stateless JWT with 15-minute expiry	Server-side sessions	Serves the web UI and a future Android client without a session store	No instant revocation; bounded to 15 minutes
ADR-05	In-process cache (Caffeine)	Redis	Single instance, reference data only, no extra container; correctness never depends on the cache	Not shared between instances (scale path in 5.6)

ID	Decision	Alternatives considered	Rationale	Trade-off
ADR-06	In-memory token-bucket limits plus Nginx edge limits	Redis-backed limiter; edge limits only	Two layers with no new infrastructure; per-user keys need the application layer	Buckets reset on restart and are not shared
ADR-07	React single-page app served by Nginx	Server-rendered pages; plain JavaScript	One API for web and future mobile; live boards need client-side state	Extra build tooling; plain JavaScript is the fallback
ADR-08	Order-row lock to serialise READY	SERIALIZABLE isolation; advisory locks; a counter column	The smallest mechanism that closes the lost-READY race and is easy to explain	Bump, recall and cancel on one order serialise, which is fine at this scale
ADR-09	After-commit events for push; in-transaction call for audit	Events for both; an outbox	Screens never see rolled-back data; audit must be atomic with the change	At-most-once push, so clients resynchronise
ADR-10	Pay first, then kitchen	Pay later	Matches the counter flow of a QSR and gives a clean state model (BR-07)	No dine-in tab flow
ADR-11	REST with OpenAPI	GraphQL	Simple, cacheable, matches the SRS and tooling	More endpoints
ADR-12	Flyway; one baseline replaced until the first demo build	Hibernate auto-DDL; Liquibase	Reviewable SQL and fast early iteration	Discipline needed after the first demo

ID	Decision	Alternatives considered	Rationale	Trade-off
ADR-13	Idempotency by insert-first unique key	Cache-based de-duplication	Correct under concurrency; reuses PostgreSQL's unique-index blocking	An extra table and a purge job
ADR-14	Fetch-based SSE client for authenticated streams	Token in the query string; cookies	Keeps tokens out of URLs and logs	A little more client code

15. Traceability and proposed SRS changes

15.1 Traceability

SRS items	Addressed in section
FR-AUTH-01 to 07, NFR-SEC-01 to 05	10
FR-MENU-06, NFR-PERF-03	5.3, 5.5, 8.3
FR-ORD-10, BR-15, NFR-CONC-01	6.2, 7.1
FR-PAY-02, FR-PAY-03	6.2, 7.2
FR-KDS-05, FR-BRD-02, NFR-PERF-02	9
FR-KDS-06, FR-KDS-07, NFR-CONC-03 and 04	6.2, 6.3, 7.3
FR-INV-02, NFR-CONC-02	6.2
FR-RPT-01 to 06, NFR-PERF-04	8.4, 8.5
FR-AUD-01 to 04	6.2, 8.1
FR-SYS-01 to 05, NFR-OBS-01 and 02	5.1, 12
NFR-PERF-01	5.4

SRS items	Addressed in section
NFR-REL-01 to 03	6, 8.6, 11
NFR-MNT-01 to 03	4.2, 13.3
NFR-POR-01, NFR-DOC-01	13
NFR-USE-01 and 02	Screen design in documents 5 and 6; not architectural

15.2 Proposed SRS changes (for SRS v0.2, through change control)

1. Add error code SERVICE_BUSY (HTTP 503) for pool exhaustion and lock timeouts.
2. FR-AUTH-06: state that lockout counters are persisted on the user and that the LOGIN rate limit applies in addition.
3. Add NFR-SEC-06: rate limits shall sit well above the design workload, and the load test shall show no 429 responses at that workload.
4. Add NFR-PERF-05: report endpoints shall be limited to 2 concurrent queries so they cannot starve order creation.
5. FR-KDS-05: add that authenticated streams close when the access token expires and clients resynchronise on reconnect.
6. FR-MENU-06: add ETag support, returning 304 when the menu is unchanged.
7. Record OI-02 and OI-03 as closed by this document.

16. Low-level design appendix

16.1 Package structure

```
com.kitchenflow
  KitchenFlowApplication
  platform/    error, correlation, ratelimit, cache, sse, scheduling,
security
  auth/        api, application, domain, infrastructure
  menu/        api, application, domain, infrastructure
  inventory/   api, application, domain, infrastructure
  order/       api, application, domain, infrastructure, spi (TicketGateway)
  payment/     api, application, domain, infrastructure
  kitchen/     api, application, domain, infrastructure
  board/       api, application
  report/      api, application, infrastructure
  audit/       api, application, domain, infrastructure
  config/      api, application, domain, infrastructure
```

16.2 Key components

Component	Module	Responsibility
GlobalExceptionHandler	platform	Maps exceptions to the error envelope and HTTP codes (SRS section 10)
CorrelationFilter	platform	Accepts or generates the correlation ID and fills the logging context
RateLimitFilter, RateLimitPolicy	platform	Resolve class and key, consume a bucket, answer 429 with Retry-After
ReportBulkhead	platform	Semaphore with 2 permits around report queries
SseHub, SseSubscriber	platform	Subscriber registry, bounded queues, heartbeat, cleanup, connection caps
MenuCache	menu	Loading cache and ETag version; evicts on MenuChanged
OrderController, OrderService	order	Use cases: create, edit, cancel, complete, search

Component	Module	Responsibility
PricingService	order	BigDecimal line pricing and GST; pure and unit-tested
TokenService	order	Allocates tokens from the daily counter row
IdempotencyService	order	Insert-first key handling, hash check, replay
OrderStateMachine	order	Transition table with roles and guards
OrderWorkflow	order	Compare-and-set transitions used by payment and kitchen
UnpaidOrderJob	order	Cancels timed-out unpaid orders using SKIP LOCKED
TicketGateway	order (spi)	Interface to create and cancel tickets; implemented by kitchen
StockService	inventory	Conditional decrements in item-id order; restore; adjust
PaymentService	payment	Payment recording and the payment-to-kitchen transaction
TicketService, TicketFactory	kitchen	Start, bump, recall with the order lock and READY evaluation; group lines by station
AuditService	audit	In-transaction record() and search
ReportService	report	SQL read models with statement timeout

16.3 Order state machine sketch

```
enum OrderStatus { CREATED, PAID, IN_KITCHEN, READY, COMPLETED, CANCELLED }

record Transition(OrderStatus from, OrderStatus to, Set<Role> roles) {}

final class OrderStateMachine {
    private static final List<Transition> RULES = List.of(
        new Transition(CREATED,    PAID,       Set.of(CASHIER, MANAGER)),
        new Transition(PAID,       IN_KITCHEN, Set.of(SYSTEM)),
        new Transition(IN_KITCHEN, READY,      Set.of(SYSTEM)),
        new Transition(READY,      IN_KITCHEN, Set.of(KITCHEN, MANAGER)),
```

```
        new Transition(READY,      COMPLETED,  Set.of(CASHIER, MANAGER)),
        new Transition(CREATED,    CANCELLED,  Set.of(CASHIER, MANAGER,
SYSTEM)),
        new Transition(IN_KITCHEN, CANCELLED,  Set.of(MANAGER)));

    void check(OrderStatus from, OrderStatus to, Role actor) {
        boolean allowed = RULES.stream().anyMatch(t ->
            t.from() == from && t.to() == to && t.roles().contains(actor));
        if (!allowed) throw new InvalidTransitionException(from, to);
    }
}
```

16.4 Bump sketch (lock order first)

```
@Transactional
public TicketView bump(long ticketId, long expectedVersion, Actor actor) {
    long orderId = tickets.orderIdOf(ticketId);          // plain read
    Order order  = orders.lockForUpdate(orderId);         // SELECT ... FOR
UPDATE, order first
    Ticket t     = tickets.get(ticketId);                 // read after the
lock
    access.checkStation(actor, t.stationId());
    if (t.version() != expectedVersion) throw new StaleVersionException(t);
    t.transitionTo(TicketStatus.DONE, clock.instant());   // state machine
guard
    if (order.status() == IN_KITCHEN && tickets.countOpen(orderId) == 0) {
        orderWorkflow.markReady(order, actor);            // compare-and-set
plus audit
    }
    audit.record(actor, AuditAction.TICKET_BUMPED, t.id(), order.id());
    events.publish(new TicketChanged(t));                 // delivered after
commit
    return TicketView.of(t);
}
```

16.5 Conventions

- DTOs are Java records; entities never leave the application layer.
- Money is a value object over BigDecimal with scale 2 and HALF_UP rounding.
- Time comes from an injected Clock, so timers and business-day logic are testable.
- Domain exceptions map to error codes in one place.

- Every public service method that changes state declares its transaction and the roles allowed to call it.

17. Risks, open items and next step

17.1 Risks

Risk	Impact	Mitigation
Infrastructure work (Nginx, observability) crowds out core features	Must features arrive late	Add them only in the polish phase; phases 1 to 3 run the application and database only
Over-engineering is hard to defend in an interview	Lost credibility	Every component has an ADR; scale-out items are documented, not built
SSE and proxy pitfalls	Delayed or lost updates	Proxy settings in 9.6, the 2-second polling fallback and the reconnect test AT-09
Concurrency bugs go unnoticed	Wrong orders under load	Concurrency tests repeated 50 times and the mutation check (6.6)
Learning load (Spring Security, JPA locking, React, Docker, Nginx)	Schedule slips	Learning map in SRS section 13; plain-JavaScript UI fallback; phase gating
Limits set too tight for the demo	False 429 responses	Limits are 12 times above the design workload; the loadtest profile is for k6 only
Cache and limiter design is misunderstood	Weak interview answers	Be able to say why in-process, what is never cached and how stale reads are prevented

17.2 Open items

ID	Item	When
AI-01	Pin exact versions (Java, Spring Boot, PostgreSQL, Nginx, React, Bucket4j, Caffeine)	Phase 0
AI-02	Confirm React or the plain-JavaScript fallback after a one-day spike	Start of phase 2
AI-03	Decide whether to include Prometheus and Grafana or stop at Actuator metrics and logs	Phase 4
AI-04	Apply the changes in 15.2 to the SRS as v0.2	Before document 5
AI-05	Your deadline, which sets how much of sections 5 and 12 is built in time	Roadmap review

17.3 Next step

Document 4, the Database Design: DDL-level schema for every table, constraints and indexes, EXPLAIN targets, the seed-data and 100,000-order generator design, and the Flyway baseline plan. It builds directly on sections 6 and 8 of this document.

Version	Date	Author	Change
0.1	29 October 2026	Rathi	First draft